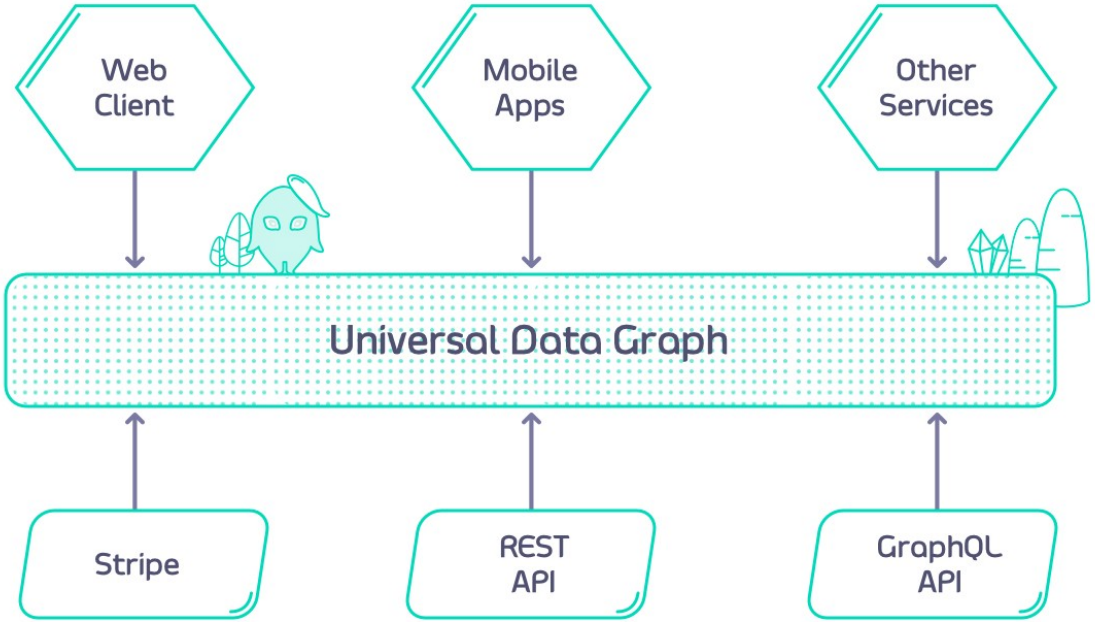
Module 3

Universal Data Graph

What is the Universal Data Graph (UDG)?

- Combine multiple APIs into one universal GraphQL interface
- Query multiple APIs with a single request
- No need to build your own GraphQL server — just configure your existing REST APIs
- Becomes the central integration point for internal and external APIs
- Inherits all Tyk capabilities: security, middleware, and governance



Currently supported DataSources:

- REST
- GraphQL
- SOAP (through the REST datasource)
- Kafka

Key Concepts - DataSources

- Resolvers vs. DataSources
  - Resolvers: Functions that return data for a field (typical in GraphQL)
  - DataSources: Config-driven way to fetch data for fields
    - No code required — just configuration

- 🔗 Types of DataSources in Tyk
- Internal:

  REST/SOAP APIs already managed in Tyk

  ➜ Use middleware to validate and transform data

- External:

  APIs not (yet) managed in Tyk

  ➜ Easily included in your data graph

  ➜ Can transition to internal when needed

Key Concepts - Arguments

- GraphQL Arguments → REST Calls

  Schema example:

```
type Query {
    user(id: Int!): User
}

type User {
    id: Int!
    name: String
}
```

  Goal: Map the id argument in a GraphQL query to the correct REST API path

Key Concepts - Arguments

Injecting Arguments into REST Paths

- Templating REST URLs
- In the “Configure Data Source” tab:

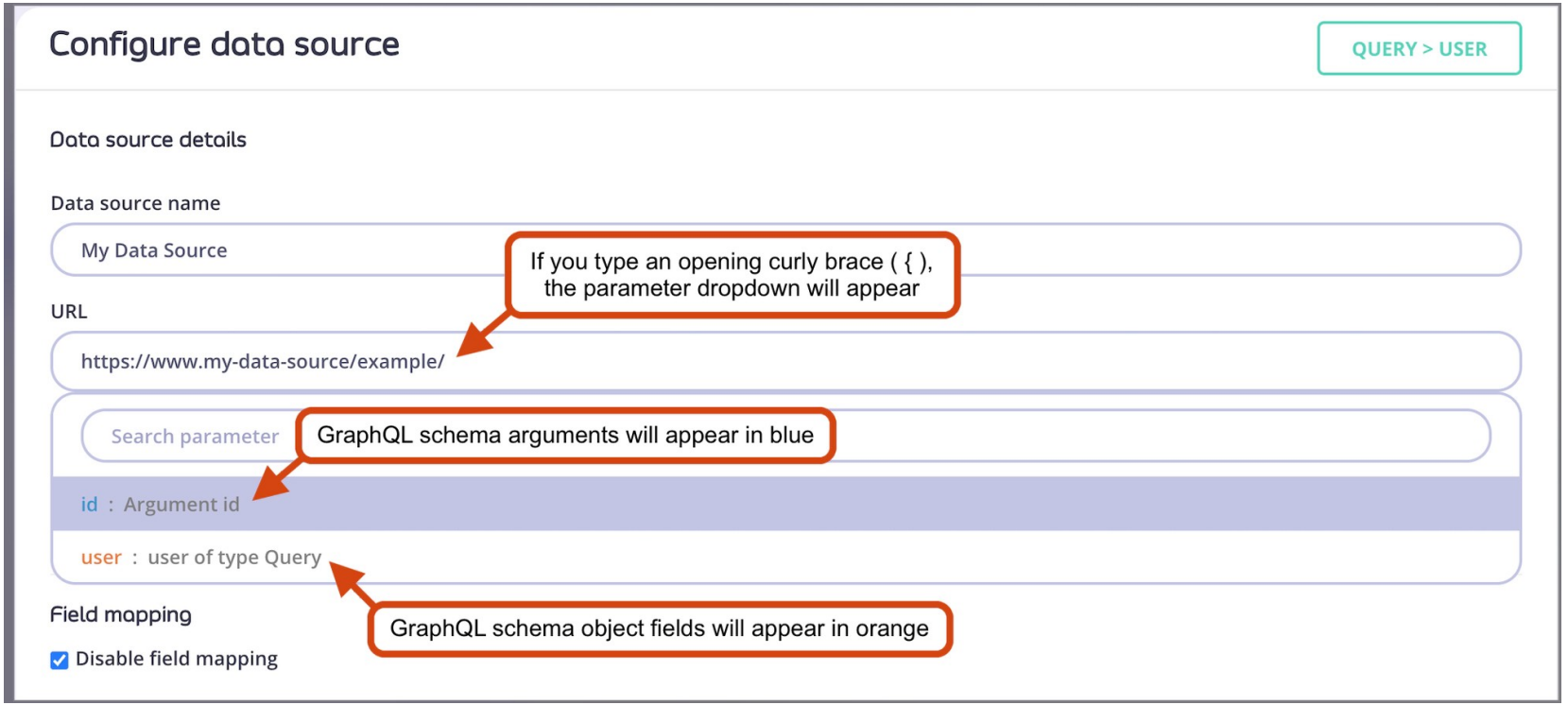
  Use this template in the path field:

  ```
  https://example.com/user/{{ .arguments.id }}
  ```

  Type `{` to access a dropdown of available fields and arguments.

  Tip: You can dynamically inject any GraphQL argument or field this way.

Key Concepts - Arguments



Key Concepts - Field Mappings

Field Mappings – When Are They Needed

Automatic Field Mapping

- When the GraphQL schema mirrors the REST API response, no manual field mapping is required.

Example REST response:

```
{
  "id": 1,
  "name": "Martin Buhr"
}
```

Matching GraphQL schema:

```
type Query {
    user(id: Int!): User
}

type User {
    id: Int!
    name: String
}
```

Key Concepts - Field Mappings

Field Mappings – Handling Different Field Names

- If the REST response uses a different field name, field mapping is required.

Example REST response:

```
{
  "id": 1,
  "user_name": "Martin Buhr"
}
```

GraphQL schema:

```
{
  "id": 1,
  "user_name": "Martin Buhr"
}
```

In this case, manual mapping is needed:

- Uncheck "Disable field mapping"
- Set `name` field path to `user_name`

- Nested fields can use dot notation: `user.full_name`

Key Concepts - Reusing response fields

Chaining Data Across APIs

You can use data from one API response to call another API.

Example APIs:

- People List: https://people-api.dev/people
- Person Details: https://people-api.dev/people/{person_id}
- Driver Licenses: https://driver-license-api.dev/driver-licenses/{driver_license_id}

Key Concepts - Defining Data Source URLs

Static and Dynamic URL Templates

- **Query.people**

Static URL for retrieving the list of people:

```
https://people-api.dev/people
```

- **Query.person**

Uses the id argument dynamically in the URL:

```
https://people-api.dev/people/{{.arguments.id}}
```

- **Person.driverLicense**

Uses data from the parent object via .object placeholder:

```
https://driver-license-api.dev/driver-licenses/{{.object.driverLicenseID}}
```

Reminder:

- Use `.object` to reference fields from the parent object.
- Use `.arguments` to reference query arguments.

Key Concepts - Defining Data Source URLs

GraphQL Query:

```
{
  people {
    id
    name
    age
    driverLicense {
      id
      issuedBy
      validUntil
    }
  }
}
```

Tyk

Response Example:


```
{
  "data": {
    "people": [
      {
        "id": 1,
        "name": "John Doe",
        "age": 40,
        "driverLicense": {
          "id": "DL1234",
          "issuedBy": "United Kingdom",
          "validUntil": "2040-01-01"
        }
      },
      {
        "id": 2,
        "name": "Jane Doe",
        "age": 30,
        "driverLicense": {
          "id": "DL5555",
          "issuedBy": "United Kingdom",
          "validUntil": "2035-01-01"
        }
      }
    ]
  }
}
```

UDG Header Management

Two levels of header control:

Global Headers

- Defined in `graphql.engine.global_headers`
- Sent to all data sources
- Can include request context variables

Data Source Headers

- Defined in `graphql.engine.data_sources[].config.headers`
- Specific to individual data sources
- Also supports context variables like JWT claims

```
{
  "global_headers": [
    { "key": "global-header", "value": "example-value" },
    { "key": "request-id", "value": "$tyk_context.request_id" }
  ]
}
```

UDG Header Management

Data Source Headers and Context Variables

Example - Data Source Header Config:

```
{
  "headers": {
    "data-source-header": "data-source-header-value",
    "datasource1-jwt-claim": "$tyk_context.jwt_claims_datasource1"
  }
}
```

Key Notes:

- Defined per data source
- Ideal for individual API authentication needs
- Can also access JWT claims and request context values

Tyk

UDG Header Management

Header Precedence Rules

When header keys overlap, Tyk applies priority:

Header name	Header value	Defined on level
example-header	data-source-value	data source
datasource1	$tyk_context.jwt_claims_datasource1	data source
request-id	$tyk_context.request_id	global

Data Source header overrides Global header if both define the same key.